
How to Connect a Client

Every way to join a JDSS network - CLI, Python SDK, ATAK and raw wire

JDSSArrow - a SOLID, pluggable reference implementation of the Joint Dismounted Soldier System (NATO STANAG 4677, specified in AEP-76, 5 volumes), with a web configuration and monitoring application.

Reference / educational software implementing a published NATO interoperability standard. Not accredited or production tactical software.

1. The join contract

To join a JDSS network a client must agree on four things with the running node: the network id, the multicast endpoint, the codec, and the security (PSK). The live values for the running node are returned by GET /api/connect and shown, ready to copy, in the Connect a Client tab of the dashboard.

Parameter	Meaning
network_id	Logical coalition network name; must match on every node.
multicast_group : port	Where frames are sent; derived from network_id if not set.
codec	Wire encoding of the JDSSDM message (xml json arrow).
security + psk	psk = HMAC-SHA256 over the payload, keyed by the coalition PSK.
repeat	How many times each frame is sent for reliability (receivers dedup).
policy_authority	If set, the node owning the coalition admit/deny policy.

There are four ways to connect a client. Pick the one that matches the client.

2. Method 1 - National soldier system / CLI node

The simplest path: drop a YAML config and run the bundled node. It joins over real UDP multicast and starts beaconing Presence.

```
# node-x.yaml - a new client joining this JDSS network
identity:
  node_id: node-x
  callsign: XRAY-9
  nation: XXX
  role: rifleman
plugins:
  codec: xml
  transport: udp
  security: psk
network:
  network_id: coalition-alpha    # must match the network you are joining
  psk: shared-coalition-secret   # must match the coalition PSK
  repeat: 3
```

```
# install, then run the node - it joins the multicast group and beacons
uv pip install -e ".[dev]"
jdssarrow run --config node-x.yaml --casevac
```

If the network has a policy authority, also add its public key so the node can verify coalition policy updates:

```
connections:
  policy_authority: node-a
  authority_public_key: <authority Ed25519 pubkey hex>
```

3. Method 2 - Custom client via the Python SDK

For a bespoke client, drive a gateway directly with the SoldierNode API. Build a GatewayConfig, start the node, then call the high-level message methods (identify, presence, report_contact, request_casevac, chat).

```
import asyncio
from jdssarrow.config.models import (
    GatewayConfig, NodeIdentity, PluginSelection, NetworkConfig,
)
from jdssarrow.gateway.gateway import JdssGateway
from jdssarrow.gateway.node import SoldierNode

cfg = GatewayConfig(
    identity=NodeIdentity(node_id="node-x", callsign="XRAY-9"),
    plugins=PluginSelection(codec="xml", transport="udp", security="psk"),
    network=NetworkConfig(network_id="coalition-alpha",
                           psk="shared-coalition-secret"),
)

async def main():
    node = SoldierNode(JdssGateway(cfg))
    await node.start()
    await node.identify()
    await node.presence(50.85, 4.35, battery_pct=90) # appears on every peer's COP
    await node.report_contact(50.86, 4.36, "dismounted patrol")
    await asyncio.sleep(2)
    await node.stop()

asyncio.run(main())
```

4. Method 3 - ATAK / Cursor-on-Target (via a bridge)

ATAK speaks Cursor-on-Target (CoT) XML over the TAK protocol, not JDSSDM. A bridge runs a JDSS node and joins ATAK's bearer, translating both ways. The bundled bridge (jdssarrow.bridges.atak) supports either ATAK mesh SA multicast or a TAK Server over TCP with optional mutual TLS. It is loop-safe and manages stale tracks.

```
# ATAK mesh SA (multicast 239.2.3.1:6969)
jdssarrow bridge atak --config examples/node-c.yaml

# ...or a TAK Server over TCP / mutual TLS
jdssarrow bridge atak --config examples/node-c.yaml \
  --tak-server tak.example.mil:8089 --tak-tls \
  --tak-cert client.pem --tak-key client.key --tak-cacert ca.pem
```

CoT <-> JDSS mapping

Cursor-on-Target	JDSS message
a-f-* (friendly)	Presence
a-h / a-u / a-n-* (hostile/unknown/neutral)	ContactSighting (with APP-6(D) SIDC)
GeoChat (b-t-f)	Chat
markers / drawings (u-d-*, b-m-p-*)	Overlay graphics
medevac 9-line (b-r-f-h-c)	CasevacRequest

A node's Presence maps to a stable CoT uid with a stale window, so ATAK keeps one moving icon and auto-expires it when beacons stop; the bridge also sweeps silent tracks (emitting a CoT delete) and drops already-stale inbound CoT. Point ATAK at the same bearer and its tracks appear on the coalition COP while coalition traffic appears on the EUD.

Connect ATAK directly to JDSSArrow (built-in TAK server)

JDSSArrow can itself act as the TAK server, so ATAK connects straight to this node with no OpenTAKServer or FreeTAKServer in the middle. In the web console open Configuration -> Serve ATAK / EUDs, set a port (default 8087) and click Start server. The node then listens for EUD connections and bridges CoT<->JDSS for each one; every connected device shows up as its own coalition peer.

- In ATAK: Settings -> Network Preferences -> Network Connections -> TAK Servers -> + and add this node's address and the port, as TCP (no SSL). The panel shows the exact address.
- The device must be able to reach the JDSSArrow host (same LAN, or routed/VPN).
- Under Docker/NAT the auto-detected address is the container's internal IP, which EUDs cannot reach. Set the 'Advertised host' field to the host's LAN IP (or a public IP/DNS), and publish the port from the container (deploy.sh publishes EUD_PORT, default 8087; or docker run -p 8087:8087). Open it on the firewall for off-LAN clients.
- Once connected, ATAK's position appears on the coalition COP and coalition traffic streams to the EUD as CoT; the panel lists connected EUDs live.

Connecting to OpenTAKServer (OTS)

The same bridge connects to OpenTAKServer, which aggregates EUDs over a TCP/SSL stream of CoT XML. OTS accepts legacy XML CoT (TAK protocol version 0), which is exactly what the bridge speaks - no protobuf negotiation is required. Two paths work:

OTS port	Auth	How the bridge connects
8088 (TCP, plaintext)	anonymous	Simplest. OTS relays CoT from a plain TCP client with no authentication.
8089 (SSL streaming)	client certificate	Present a client cert whose CommonName is a registered OTS user; OTS authenticates it by cert alone (no username/password message needed).

```
# 1) Plaintext TCP on OpenTAKServer's port 8088 (anonymous)
jdssarrow bridge atak --config examples/node-c.yaml \
  --tak-server ots.example.org:8088

# 2) SSL on port 8089 with a client cert enrolled for an OTS user
#   (export the .p12 from OTS and split it into PEMs, or use your enrollment certs)
jdssarrow bridge atak --config examples/node-c.yaml \
  --tak-server ots.example.org:8089 --tak-tls \
  --tak-cert ots-client.pem --tak-key ots-client.key --tak-cacert ots-ca.pem
```

- OTS's SSL port requires authentication: a client cert whose CN names a known OTS user authenticates by cert; an unknown/unenrolled cert connects but its CoT is ignored.
- The bridge's HMAC/PSK security is JDSS-side only; the CoT carried to OTS is standard XML, so ATAK/WinTAK/iTAK clients on the same OTS see the coalition tracks and vice versa.
- Verified against OpenTAKServer's own EUD-handler framing and cert-auth logic in the test suite (tests/test_opentakserver_interop.py).

From the web console - no CLI

The Configuration tab has a CoT/TAK Server Connections panel that manages these connections (CRUD) from the browser, with live status. To add OpenTAKServer:

- Plaintext: add a server with host + port 8088, TLS off - it connects immediately.
- TLS (8089): tick TLS and import a .p12 / .pfx (client cert + key + CA) with its password right in the browser. The backend decrypts it (POST /api/servers/pkcs12) into PEM material stored on the connection - no file paths, no shell. The cert's CommonName must be a registered OTS user.
- Use Test connection to probe before saving; edits and enable/disable apply live and persist to the config file.

5. Method 4 - Any language, raw wire format

Any language can join by producing the JDSS frame and authentication directly. The frame layout is:

```
frame = magic("JDSS",4) | version(1) | codec-name | security-name | len(4, big-endian) |
payload
payload = security.protect( codec.encode(JdssMessage) )
```

Steps:

1. build a JDSSDM message (header + typed body) and encode it with the chosen codec
2. append/verify HMAC-SHA256(payload, key = coalition PSK) [security = "psk"]
3. send the frame to the UDP multicast group:port (send it `repeat` times)
4. receivers dedup by (originator_id, message_id)

The seven message types available on the network are listed below.

Message type	Body summary
Presence	position (lat/lon), status/battery
Identification	callsign, unit, nation, role
ContactSighting	observed location + APP-6(D) symbol id
Sketch	ordered list of points
Overlay	list of graphic control measures
CasevacRequest	casualty location + urgency (9-line)
Chat	free text + optional recipient

6. Why a client might be rejected

A joining client's traffic is validated on ingest. If it is dropped, the reason is one of the audit tokens below (visible in the Logs tab and in Prometheus counters). Checks run in order: framing, then security, then codec/schema, then policy, then capability.

Reason	Cause / fix
framing	Not a valid JDSS frame (bad magic/length). Check the wire format.
security	HMAC verify failed. The PSK does not match the coalition PSK.
codec	Payload did not decode/validate against JDSSDM. Check the codec + schema.
policy	The peer is blocked by local or coalition admit/deny policy.
capability	The message type is disabled for receive on this node.
duplicate	A repeat of an already-seen (originator_id, message_id) - expected.

7. Verifying the connection

- Dashboard -> Connected Peers: the new node appears with its callsign, nation and role.
- Dashboard -> Live Message Feed: its Presence/Identification stream in.
- Connections & Policy -> Connection Matrix: its column fills in as nodes accept it (a rejected node's column stays all-zero).
- Logs -> Message Log: filter by disposition = rejected to see any drop reasons.